Abdelrahman Rashed 900231248, Ziad Khairaldin 900225292, DD2 (CSCE 3304) Fall 2026, group gb2.

DD2 Project 1, Option 2 - Finding Cells Worth Adding

Option 2 (three-student group, real cells from the course generator). AI used as a development aid. Repository github.com/Gad101/cellgap, tag final; code and results at commit 9c0dc73 (the tag adds only this report). Every number below comes from a file in results/ that the flow wrote; the file is named next to the number.

part 1: introduction

The problem, and what we built.

HD already has most small single-stage functions. The question is which small functions real designs compute again and again, whether HD lacks them, and whether a cell for them makes the designs smaller when Yosys maps them. We built cellgap, a deterministic tool (no AI model runs in it) that does the 12 steps of handout section 4. make all runs steps 1-9, make final steps 10 and 12 and the final tables, make real-cells and make real-lib step 11, and make demo IN=<design> TOP=<top> runs the whole flow on one design (the interview run). Result: an estimated saving of 0.17%, which drops to 0.01% with real cells. Both figures are measured and checked for equivalence, and the drop is explained in part 5.

part 2 step 1:

Extract the AIG, and the graph after ABC restructuring.

src/make_aig.sh: read_verilog, synth -flatten, async2sync, dffunmap, aigmap, write_aiger -zinit. With -zinit the AND counts equal handout Table 2 for all six required designs. The after-ABC graph is strash; dch; strash (src/abc_after.sh; the last strash removes dch's dangling choice nodes) and is checked equal to the original with ABC cec.

part 2 step 2:

Enumerate cuts of up to 4 inputs with a truth table each, and check them by simulation.

src/cuts.py, our own bottom-up enumerator, with no cut limit. Every cut's 16-bit table is re-simulated for all leaf combinations, and the leaves are checked to separate the root from the inputs. The check passes on all 16 graphs (8 designs, before and after ABC).

part 2 step 3:

Classify up to input order and output inversion; count NPN as well.

src/tt.py precomputes canonical forms for all 65,536 tables: P (input order), PO (input order and output inversion) and NPN. PO is the class we cost and compare; NPN is reported only for reference (part 5).

part 2 step 4:

Cost every class: stages, inverters, transistors, at most 3 in series.

src/costmodel.py, src/cmos.py: the cheapest of one static CMOS stage, the same with input/output inverters, or a Shannon split into stages. Every cost carries a formula that is re-simulated and must reproduce its class (3,984 of 3,984).

part 2 step 5:

Check redundancy against HD (section 5) and drop what HD already has.

src/redundancy.py: every class against every single-output combinational cell of the full HD Liberty, all drives. The result is Exists, Exists with an inverter, Contained or New, with Table 3's area exceptions. The checker reproduces Table 4 (part 3).

part 2 step 6:

Estimate the area with a model fitted to HD's x1 cells, and report its error.

Width in sites against transistor count, one line for single-stage cells and one for multi-stage (results/area_model.json):

one-stage: sites = 1.053 + 0.553 T (13 cells), mean error 0.62 sites (15.9%), max 1 site (a21oi_1)

multi-stage: sites = 2.512 + 0.461 T (45 cells), mean error 0.98 sites (14.9%), max 3 sites (or4_1)

nand4_1 and nor4_1 are left out because they have 4 transistors in series.

part 2 step 7:

Rank the candidates by estimated saving.

src/candidates.py. The saving of one occurrence = (ANDs in its exclusive cone, capped at the smallest cone seen for that function) x the design's baseline area per AND - the new cell's estimated area. Non-overlapping occurrences are picked per design, and the classes are ranked by the total. This is a guess, as the handout says; steps 9-10 measure the real saving.

part 2 step 8:

Write the requests (Appendix A) and the Liberty file.

src/write_cell_requests.py: one request per candidate, refused unless the expression and truth table agree. src/write_liberty_with_new_cells.py: hd_reduced.lib plus the new cells in the section 6 format. Yosys must import 189 + N cell types, and every cell's function must equal its request.

part 2 step 9:

Map every design with and without the new cells; check with cec and Tutorial 4's testbenches.

The commands of handout section 2 with scripts A and B, only the library changing; the baseline equals Table 1. Every netlist is checked against the RTL with ABC cec (no flip-flops) or dsec (with flip-flops). SPM and mul also run Tutorial 4's gate-level testbenches with Icarus Verilog.

part 2 step 10:

Select the final set greedily, at most 10 cells.

src/select_final_set.py: in rank order, add the next candidate, remap the six required designs, and keep it only if the best-of-both total drops by more than 0.05 um2. Stop at 10 cells or at the end of the list.

part 2 step 11:

Send the final set to the cell generator and use the real cells.

src/run_cellweaver.py runs cellweaver on each final request: Magic DRC, Netgen LVS, abutment, extracted-RC characterization and OpenSTA validation. src/real_lib.py swaps each real cell group in for its estimate and reruns the import and function checks. cells/<cell>/ holds the GDS, LEF, Liberty and Verilog, and the Appendix A response.

part 2 step 12:

Make one new cell 30% smaller and remap.

src/sensitivity.py: the most-used final cell, everything else unchanged.

part 2: worst delay

Report the worst delay from OpenSTA for each design.

src/worst_delay.py, with the same constraints for every netlist: a 10 ns clock on the port that drives the flip-flops, all other ports at 0 ns, no load. The worst delay is the worst data arrival time, with reset recovery paths left out.

part 3: verification

Every check, and the broken case that shows it works.

make test runs 63 tests in the pack image; 2 more need Icarus and run on the host.

Request: expression and truth table agree

tests/broken/wrong_expression.json (0x077E for the 0x077F formula) is refused.

Cut table equals simulation

wrong_cut_table.json (one flipped row) is caught.

Leaves really cut the node

not_a_cut.json (a dropped leaf) is caught.

Redundancy self-test

All 258 HD cells with up to 4 inputs come out Exists, and Table 4 is reproduced. With hd_missing_cells.json (HD without a22oi), Table 4 no longer matches.

Cost formula computes its class

A formula for another function (0x0707 with !(A&B)) is rejected.

After-ABC graph equals the original

csa8 with one AND input inverted is rejected by cec.

Liberty loads, 189 + N cells, functions match

An unclosed quote is rejected. A missing brace is read by Yosys 0.33 but only 190 cells are imported, so the count check fails. A function naming a missing pin is reported.

Every mapping equals the RTL

One nand2_1 swapped for nor2_1 is rejected in mul (cec) and in cpu8 (dsec).

Gate-level testbenches

The same swap fails mul_exhaustive_tb.

Greedy rule

A cell that grows the total, and a gain below rounding, are both dropped.

Final table

A failed equivalence shows as FAIL. A real Liberty with a missing or unbalanced cell group is refused.

Calibration.

Table 1 is reproduced exactly: all 24 areas at 0.000% (results/table1.md, native Yosys 0.33).

The AIG AND counts equal Table 2.

Table 4 is reproduced: 24 / 20 / 38 / 3,822; 9 New one-stage; read-once 8 = 6 Exist + 2 Contained. One row differs, 6 against 5 (part 6).

part 4 step 1: designs, cuts and classes

AIG size, cuts and classes per design, before and after ABC (results/design_stats.csv).

Design	ANDs	Cuts	P classes	NPN
cpu8	1,265 / 1,109	10,725 / 9,007	281 / 233	63 / 59
dsp8	1,973 / 1,532	16,445 / 13,032	268 / 252	54 / 50
arlet6502	3,221 / 2,699	27,091 / 20,438	312 / 287	51 / 60
uart_apb	4,673 / 2,971	39,290 / 17,934	319 / 180	45 / 38
risc8	7,750 / 6,592	57,178 / 44,144	276 / 223	45 / 47

Design	ANDs	Cuts	P classes	NPN
fir10	8,795 / 6,937	92,196 / 75,435	232 / 212	43 / 43
spm	953 / 540	10,563 / 3,179	109 / 51	29 / 22
mul	774 / 512	8,221 / 5,356	242 / 196	52 / 36

Each cell gives the value before / after ABC. Over all eight designs: 362 classes up to input order and output inversion (316 after ABC) and 91 NPN classes (86) (results/class_counts.csv). HD's own 258 small cells fall into 29 PO classes (results/hd_class_table.md).

part 4 step 2: redundancy report

Every candidate with its Table 3 class and the HD cell it matches (results/redundancy_report.csv).

556 classes occur in real cuts:

Table 3 class	Classes seen	Kept
Exists	53	0
Exists with an inverter	28	23 (estimated smaller than the HD cell + inv_1)
Contained	30	28 (estimated smaller than the HD cell)
New	445	445

Examples:

0x000F is nor2 (Exists).

0x00EE is o21ba (Exists).

0x07FF, !(D & (C | A&B)), is Contained in a32oi and kept (7.51 against 8.76 um2).

0xEEFA is Contained in mux4 and kept (12.51 against 22.52 um2).

part 4 step 3: ranking and final set

The ranked candidates with estimated savings, and the final set (results/candidates.json, results/greedy_log.csv, results/final_set.json).

11 candidates have a positive estimated saving. Greedy selection remaps the six required designs for each try:

Try	Class	T / stages	Est. area	Est. saving	Total after (um2)	Decision
1	0xEEFA	16 / 3	12.51	36.74	86,282.8	drop (+37.5)
2	0x8AE6	34 / 4	23.77	25.92	86,245.2	drop
3	0x17	10 / 1	8.76	12.01	86,235.2	**keep**
4	0x077F (handout's maj4n)	12 / 1	10.01	8.25	86,235.2	drop
5	0x17FF	12 / 1	10.01	5.50	86,234.0	**keep**
6	0xBCC4	34 / 4	23.77	3.70	86,234.0	drop

Try	Class	T / stages	Est. area	Est. saving	Total after (um2)	Decision
7	0x07FF	8 / 1	7.51	2.84	86,102.6	**keep**
8-11	0xAABF, 0x0BBF, 0x2BFF, 0x0EEF			1.59 to 0.74		drop

Final set: dd2_gb2__f17_1, dd2_gb2__f17ff_1 and dd2_gb2__f07ff_1, all single-stage.

part 4 step 4: Table 5 with estimated areas

Baseline against the extended library, scripts A and B, and the best of both (results/table5_final_est.md; the per-script rows, instance counts and delays are in that file).

All 32 mappings pass cec/dsec. Best of both per design:

Design	Reduction
dsp8	0.23%
arlet6502	0.49%
uart_apb	0.07%
risc8	0.26%
cpu8, fir10, spm, mul	0.00%
Total, required designs	**86,245.3 -> 86,102.6 um2 (0.17%)**

With the same script, script B gives fir10 2.10% (241 instances), cpu8 1.75%, mul 1.41% and arlet6502 1.11%. Script A uses only f07ff.

part 4 step 5: real cells

The final set with real areas from the generator (results/real_cells/, cells/, results/real_vs_estimated_area.csv).

cellweaver built all three cells at single height. DRC, LVS, abutment in four orientations and extracted-RC characterization all pass, and OpenSTA validates each Liberty. Real against estimated area:

Cell	Estimated (um2)	Real (um2)	Real / estimated
f17	8.76	17.52	2.0x
f17ff	10.01	30.03	3.0x
f07ff	7.51	16.27	2.17x

part 4 step 6: Table 5 with real cells

The same table with the real cells (results/table5_final_real.md).

All 32 mappings pass cec/dsec. ABC almost never picks the real cells: fir10 with script B uses 2 instances, and every other mapping none. Best-of-both total: **86,245.3 -> 86,239.0 um2 (0.01%)**. The only design that gains is risc8 (0.02%).

part 4 step 7: cell-by-cell curve

Total best-of-both area after each final cell, in the order the tool chose them (results/curve.md, results/curve_real.md, and the SVGs beside them).

Cells added	Estimated (um2)	Real (um2)
0	86,245.2	86,245.2
1 (f17)	86,235.2	86,246.5
2 (+f17ff)	86,234.0	86,244.0
3 (+f07ff)	86,102.6	86,239.0

part 4 step 8: the 30% experiment

One new cell 30% smaller in the Liberty, remapped (results/sensitivity_30pct.md).

f17 goes from 8.76 to 6.13 um2. Script B then uses it much more: fir10 237 -> 403 instances (-1,187.7 um2), dsp8 27 -> 40 (-102.6). Script A never uses it. The best-of-both total moves only from 86,102.6 to 86,100.7 um2 (-1.9), because script A without the new cells is still smaller on those designs. ABC sees only the Liberty's numbers, so the result depends strongly on the area estimate for script B, and hardly at all for the best of both.

part 4 step 9: worst delay

The worst delay from OpenSTA for each design (results/worst_delay.csv, also in both Table 5 files).

Estimated cells

The largest slowdown is cpu8 B, 4.11 -> 4.24 ns (+3%). Several script B paths get faster (risc8 13.32 -> 10.98, arlet6502 8.65 -> 6.87, dsp8 5.63 -> 4.70 ns), because one complex gate replaces a chain of gates. These delays rest on the placeholder arcs of section 6.

Real cells

The arcs are measured, but the netlists barely change, so most delays track the baseline; the largest change is arlet6502 B, 8.65 -> 6.27 ns.

part 4 step 10: gate-level testbenches

Tutorial 4's testbenches on the SPM and mul netlists (results/gate_level_testbenches.csv).

24 of 24 runs pass: baseline, estimated, final estimated and final real, including the exhaustive testbench over all 65,536 multiplier input pairs.

part 5: discussion

How many 4-input classes up to input order and output inversion, how many NPN, and why the first count matters for CMOS.

362 PO classes against 91 NPN. In CMOS, swapping inputs is only rewiring, and an inverted output comes free with the dual network; an inverted input costs an inverter (2 transistors,

usually a site). NPN treats that inverter as free, so its smaller count overstates how much one cell covers.

How many candidates each class of Table 3 removed, with examples.

Exists removed 53, Exists with an inverter 5 of 28, and Contained 2 of 30; examples are in part 4 step 2. The exceptions mattered: f07ff (0x07FF) is in the final set only because its estimated area beat a32oi.

Why the saving differs between designs.

The kept cells are carry-like single-stage functions. Script B (&nf, which matches functions) uses them where MAC/carry cones repeat: fir10 B 2.10%, cpu8 B 1.75%. Script A (amap, which matches structure) never uses a cell with a repeated input, as the handout warns. So on cpu8, fir10, spm and mul, where script A without new cells is already the smaller result, the best-of-both gain is zero. Gains remain only where script B with the new cells beats script A without them: arlet6502 0.49%, risc8 0.26%, dsp8 0.23%, uart_apb 0.07%.

Why the saving disappears with real cells.

The area model is fitted to HD's hand-packed x1 cells. The generator's shared-diffusion layouts spend extra sites on routing access, so the real cells came back 2-3x the estimate. At those areas ABC rarely picks them, and 0.17% becomes 0.01%. At HD density these functions would be worth a cell; at 2-3x HD density they are not. The measurement decides, not the cut count.

Do risc8, pic16 and p16c5x, which implement one instruction set, favour the same new cells? (results/isa_compare.md)

No. Ranked on its own, risc8 has 6 candidates with a positive saving, pic16 none, and p16c5x one: 0x07FF, which is also our third final cell and risc8's rank 4. Their most frequent New classes overlap much more: pic16 and p16c5x share 9 of their top 10, and risc8 shares 4 with each. The same ISA gives similar logic, but the structure of each synthesized AIG decides which cuts exist and which of them save area.

An example of a frequent function that did not save area, and why.

0x077F, the handout's own maj4n (the carry of a carry-save adder), is frequent: script B used it 31 times in SPM alone. Yet SPM with script B grew by 1.33% with it (results/table5_estimated.md), and its greedy try saved 0.0 um2. Putting a carry in one cell breaks up the maj/xor structure that neighbouring logic shared, so ABC pays for it elsewhere. Similarly, 0xEEFA ranked first by estimate (36.7 um2) but made the total 37.5 um2 larger. A function's frequency says nothing about whether the logic around it gets cheaper; only remapping shows that.

part 6: limitations

What the area model, the Liberty file and the two scripts leave out.

Table 4, one row

"New, one stage, at most 3 in series and at most 12 T": we count 6, the handout 5. Each of our 6 has an explicit 12-transistor network within 3 in series, re-checked from its formula

(results/table4_selftest.txt). They form 3 dual pairs, so any rule that treats the two networks alike gives an even count. We report the mismatch and did not adjust the target.

Area model

Straight lines in transistor count, about 15% mean error and 3 sites at most. It also assumes HD density, which the generator does not reach (2-3x).

Cost model

An upper bound: no inverter sharing between stages and no exhaustive factoring.

Ranking

A guess by construction. The AIG's structure limits which cuts exist, and the first-ranked candidate measured worse than adding nothing.

Greedy

Each candidate is tried once, in rank order.

Liberty

Estimated cells have one scalar arc per input. Real cells have measured TT tables only: no SS/FF corners and no power.

Scripts

amap ignores cells with a repeated input, and &nf -R 1000 relaxes delay. Changing the library alone can move script A by about 1%.

Tools

Table 1 reproduces only with native Yosys 0.33; yowasp-yosys 0.33 and Yosys 0.70 give different areas. cellweaver needed its own qualified Magic and Netgen builds (container/Dockerfile).

part 7: AI use

part 8: contributions

Who did what, agreed by every member.

Abdelrahman Rashed (Person B)

Checkpoint 1: AGENTS.md, truth tables and classes, Liberty and AIGER readers, cut enumerator and its checks, redundancy checker and Table 4 self-test, Table 1.

Checkpoint 2: requests, Liberty writer, mapping, equivalence, gate-level testbenches, Table 5.

Checkpoint 3: interview demo, OpenSTA delays, real cells through cellweaver, Appendix A responses, README and reports.

Ziad Khairaldin (Person A)

Checkpoint 2: cost model, area model, candidate finder and ranking.

Checkpoint 3: greedy final set, curve, final Table 5, 30% experiment, ISA comparison, real-cell hook.

appendix: tool versions

Tool	Version
Yosys + yosys-abc	0.33 (git sha1 2584903a060), Ubuntu 24.04 apt, in the pack image dd2-layout-tools
Python / numpy	3.12.3 / 1.26.4
HD Liberty	sky130_fd_sc_hd__tt_025C_1v80.lib, open_pdks bdc9412b, sha256 e66aab4e...2c111baf
Icarus Verilog	13 (host)
Magic / Netgen / ngspice	pack image 8.3.684 / 1.5.133 / 47 with the BSIM4 fix; for cellweaver, Magic 4f53bb3 and Netgen 1.5.323 e1528a7
OpenSTA	3.1.0, built from source at f8e13cb
cellweaver	github.com/shalan/cellweaver e7fd28e, PDK with sky130_fd_pr models (same open_pdks version)